

Week-8 DL Gen-AI

L1

Memory-Based RNN Models

- RNN Challenges $\rightarrow$ Vanishing / Exploding gradients

small nets.,
gradient signal
shrinks fast

for large nets.
gradient signal
grows exponentially.

solution

memory-based architectures

(LSTM $\rightarrow$ long short term memory)

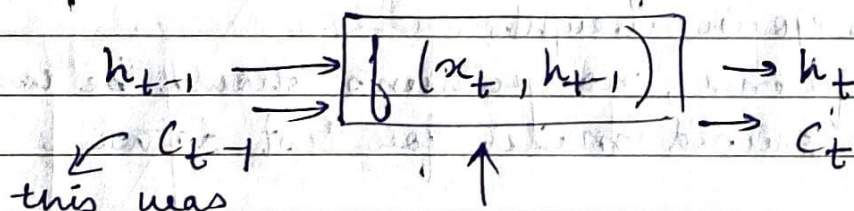
- vanilla RNNs $\rightarrow$ lookahead, no explicit

$\hookrightarrow$ not much used in practice memory.

$\hookrightarrow$ we need more paths for gradient to flow

$\hookrightarrow$ also, you can retain some of the previous calculation.

LSTMs have explicit memory, while GRU has implicit memory.



added x_t to $(p-1)$ ($p=1$)

memory is added loosely & we learn it also.

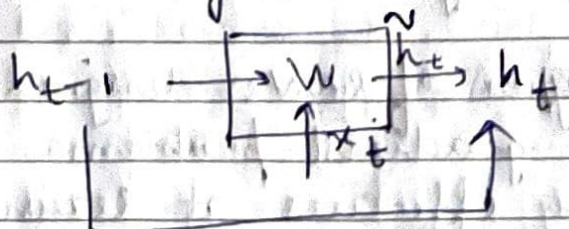
- Simplified GRU

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \bar{h}_t$$

$$z_t = \sigma(w_z [h_{t-1}, x_t] + b_z)$$

$$\bar{h}_t = \tanh(W_h[h_{t-1}, x_t] + b_h)$$

The single update gate (z_t) blends the old & new memory, no reset gate.



- GRU (Gated Recurrent Unit)

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \bar{h}_t \quad (\text{update})$$

$$\bar{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$$

[0,1] $\leftarrow$ reset gate (value)

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

GRU adds a reset gate (r_t) for selective forgetting

- Original LSTM (4x Vanilla RNN)

$$\left. \begin{aligned} h_t &= o_t \odot \tanh(c_t) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \bar{c}_t \end{aligned} \right\} \begin{array}{l} \text{extra memory} \\ \text{updates} \end{array}$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad \text{forget gate}$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad \text{input gate}$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad \text{output gate}$$

$$\bar{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \text{ candidate state}$$

3 gates & a sep. cell state (c_t) enable long-term memory.

- LSTM is types of RNNs designed to solve the vanishing gradient problem. They are particularly effective at learning long-term dependencies in sequential data!

→ dual states
→ gated mechanisms
→ gradient stability

} key differences
w.r.t. RNNs

- forget gate → decides what info. from old cell state to throw away.

→ $f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$
it learns to selectively discard past info. based on the current input & previous hidden state.

- GRU is a simpler alternative to LSTM.
2 gates → update & reset gates.

controls the short-term memory

L2 Bidirectional RNNs

- Limitations of RNN

→ std. RNN is unidirectional.

→ hidden states only captures past inputs.

→ major drawback for tasks where future context is crucial.

- Bi-RNN solves this by processing the seq. in 2 directions simultaneously.

→ It has 2 indep. RNNs, one to process the seq. $L \rightarrow R$ (forward layer) & other $R \rightarrow L$ (a backward layer).

Then concatenate the results.

- Forward pass ($\vec{H}_t$)

Backward pass ($\overleftarrow{H}_t$)

$$H_t = [\vec{H}_t; \overleftarrow{H}_t]$$

$$O_t = H_t W_{hg} + b_g$$

O_t is computed & loss is backpropagated.

eg. task of predicting a word in fill in the blanks.

→ The concatenation of $\vec{H}_t$ & $\overleftarrow{H}_t$ is input.

- Std. RNNs are designed where input & output lengths are same. How to handle tasks like MT or Summarization where lens are diff.?

- Seq2Seq model → 2 RNNs -

1. Encoder → Reads entire input. Compress whole seq. into a context vector.

2. Decoder → generates output seq. 1 token at a time, using context & previously generated tokens.

soluⁿ →

L3

Encoder-Decoder Architectures

- Std. RNNs assume I and O lengths are same.
Output len $\leq$ Input length

What to do in tasks like machine translaⁿ or summarization?

→ we do it using Enc-Dec. networks.
(Seq2Seq models)

- During training, instead of feeding the decoder's own predicⁿ from prev. step. We use the actual ground-truth token from the target sequence. This technique is $\Rightarrow$ Teacher Adv. $\swarrow$ Forcing

stabilize the training & help the model to learn the seq. efficiently, with 100% correct inputs.

- At inference time, we use previously generated tokens only. since, ground is not available.

- The enc. & dec. models are trained jointly for a seq. to seq. task:

Loss $\Rightarrow$ Cross-Entropy (classificaⁿ problem)

eg. 300,000 classes

→ calculated at each time step of the decoder's output by comparing the predicted probab. distribuⁿ over vocab.

- Masking $\rightarrow$ when you work with seq. batches with diff. lens, padding is used to make them uniform.

- Evaluaⁿ $\rightarrow$

$\rightarrow$ BLEU (Bilingual Evaluaⁿ Understudy) $\rightarrow$ std. metric for quality of the predicted sequence

by comparing it to the ground truth sequence.

eg. $\rightarrow$ Translaⁿ, summarization, captioning
 $\rightarrow$ based on the precisions of n -grams (contⁿ. sequence)

$$\text{BLEU} = \exp\left(\min\left(0, 1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}}\right)\right) \cdot \left(\prod_{n=1}^k p_n\right)^{1/k}$$

$\hookrightarrow$ as close to 1, is better

$k \rightarrow$ longest n -gram considered

$\text{len} \rightarrow$ no. of tokens in seq.

eg. $k=2$ label $\rightarrow$ A B C D E F

pred $\rightarrow$ A B

i. calculate n -gram precisions (p_n)

$$p_1 = \{A, B\}$$

$$p_1 = 1$$

$$p_2 = \{A, B\}$$

$$p_2 = 1$$

we have to penalize short seq. $\Rightarrow$ BP

$$\boxed{\text{BP} = \exp\left(1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}}\right)}$$

Brevity
Penalty

$$\text{BP}_1 = \exp(-2) \approx 0.135$$

$$\text{BLEU} = \text{BP} \cdot (p_1 \times p_2)^{1/2} = 0.135$$

- Seq2Seq models is foundation for NLP tasks where I/O lengths are not same.

- Challenges

→ when gen. the seq., goal is to find the seq. with highest conditional probability.

→ choice of picking looses its creativity.

⇓

Greedy Searcher → choose single most likely token. Fast & cheap. Not optimal.

Exhaustive Search → cal. the probab. of every single possible sequence. Guaranteed to find the optimal seq. The cost is too high.

Beam Search → beam size (k) → explore the most promising options in a limited way at each step.

→ you get $k \times |Y|$ new candidate sequences

$$Score = \frac{1}{L^\alpha} \log P(y_1 \dots y_L | c)$$

$$= \frac{1}{L^\alpha} \sum_{t'=1}^L \log P(y_{t'} | y_1 \dots y_{t'-1}, c)$$

$L \rightarrow$ len of seq.

$\alpha \Rightarrow$ HPT → controls the strength of lengths penalty

14 Sentiment Analysis - Demo

- recap of RNNs

$$D = \{w_1, \dots, w_n\} \quad f: D \rightarrow \{0, 1\}$$

$$f(D) = 1 \Rightarrow +ve$$

$$f(D) = 0 \Rightarrow -ve$$

$$\hat{y} = \sigma(w_{ny} h_n + b_y)$$

- import essential libraries
- download the dataset & load it.
- perform some initial data exploration.
- visualize the class distribution, etc.; perform the overview inspection; $\Rightarrow$ EDA.
- Data preprocessing $\rightarrow$ text normalization, punctuation removal,
- text tokenization & encoding $\rightarrow$ build vocab, turn into encoding, labelling
- prepare the dataset for NN $\rightarrow$ padding
- 2 NNs $\rightarrow$ RNNs, LSTMs for training
- configure the training parameters
 $\hookrightarrow$ Adam, LR Scheduler.
- finally, perform the evaluation checks & study the comparative analysis.